
Travl Portal

**CS 465 Project Software Design Document**

Version 1.0

**Table of Contents**



**Document Revision History**

| Version | Date | Author | Comments |
|---|---|---|---|
| 1.0 | 05/23/2025 | Andrew Torrez | Initial draft created; included Executive Summary, Design Constraints, and System Architecture View. |
| 2.0 | 06/07/2025 | Andrew Torrez | Integrated backend refactor to separate app_api module and connect MongoDB to endpoints. Added completed sequence diagram to System Architecture View section. Added completed class diagram illustrating JavaScript classes for Travlr Getaways. Added API Endpoints table documenting HTTP methods, URLs, and purpose of each endpoint. |
| 3.0 | 6/21/2025 | Andrew Torrez | Added a summary comparing the Angular project structure to the Express backend, highlighting Angular's component-based architecture and routing system versus Express's MVC structure. Explained the rich functionality of the Single Page Application (SPA), including dynamic rendering, seamless navigation, and client-side form handling, which provides a more interactive user experience than traditional server-rendered pages. Also described the testing process used to confirm the SPA's integration with the API, including using Postman and browser dev tools to verify GET and PUT requests successfully retrieve and update data in the MongoDB database. Added supporting screenshots of the Add Trip and |

| Version | Date | Author | Comments |
|---|---|---|---|
| | | | Edit/Update Trip pages to demonstrate the working UI and its connection to the API. |

**Executive Summary**

The Travlr Getaways web application is being developed to meet the business goals of a travel company that offers customizable travel packages to customers. As a software developer assigned to this project, you are responsible for building a full-stack solution using the MEAN stack — MongoDB, Express, Angular, and Node.js — that includes both a customer-facing website and a single-page administrative portal.

Customers using this application will be able to create an account, search for travel packages by location and price, and book their reservations. They will also be able to revisit the site later to review their trip itineraries, which means the application must support persistent user sessions and dynamic content rendering. On the administrative side, Travlr Getaways staff will need access to a secure, internal site where they can manage customer data, configure available travel packages, and adjust pricing.

To meet these dual requirements, the application will be structured around two primary experiences. The customer-facing side will initially be built using Node.js and Express, with Handlebars as the templating engine to render views and deliver static and dynamic content. As development progresses, the administrative interface will be implemented as a single-page application using Angular, providing a richer and more responsive environment for staff to manage content and user accounts. Both parts of the system will interact with the same backend API built in Express, which communicates with a MongoDB database that stores user information, travel packages, and booking details. This architectural decision allows for separation of concerns, reusability of data models, and flexible future enhancements.

**Design Constraints**

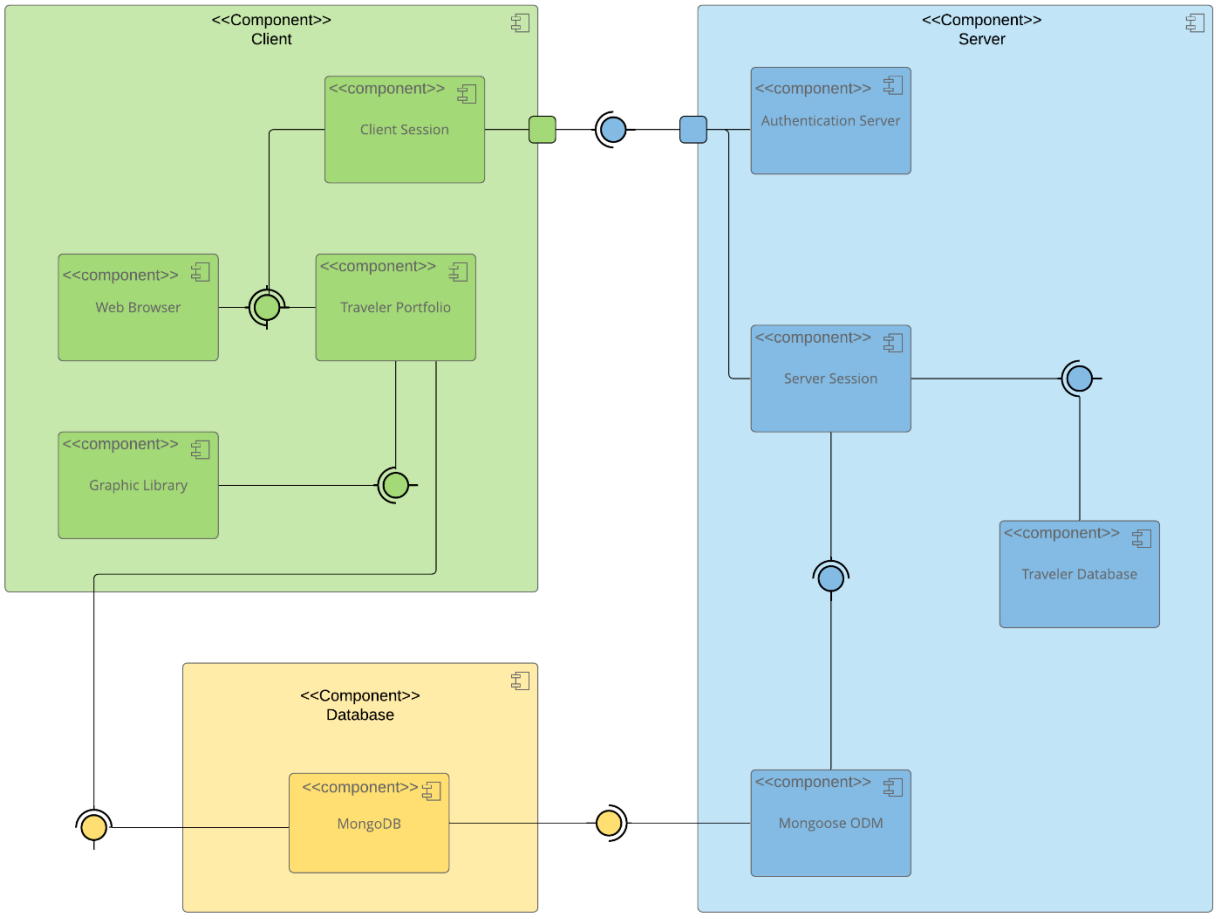
The design of the Travlr Getaways platform must take into account several key constraints dictated by both the business requirements and the technical tools chosen for the project. The most significant constraint is the need to serve two distinct user types: customers and administrators. These groups have very different needs and expectations, which means the application must clearly separate public and private functionality. From a development standpoint, this results in different user interfaces, different routing structures, and separate logic for handling permissions and access control.

Another constraint is the requirement to allow customers to search packages based on specific criteria like price and location. This means the underlying database must be designed to handle filtered queries efficiently, which will influence how the travel package documents are structured in MongoDB. Because customers will log in and return to view their itineraries, the application must persist data reliably and support user sessions. This places additional constraints on how authentication, session handling, and data validation are implemented — particularly in the shared API.

Security is also a critical design consideration. Administrative features like user management and pricing updates must be protected behind authentication and role-based access controls. This requires proper API endpoint security and a clean distinction between public and private data exposure. On top of that, the application must be modular and scalable. As the user base grows, both the customer-facing and administrative systems must be able to scale independently, which is another reason the MEAN stack — particularly the separation between Angular and the Express API — was chosen.

## System Architecture View

### Component Diagram



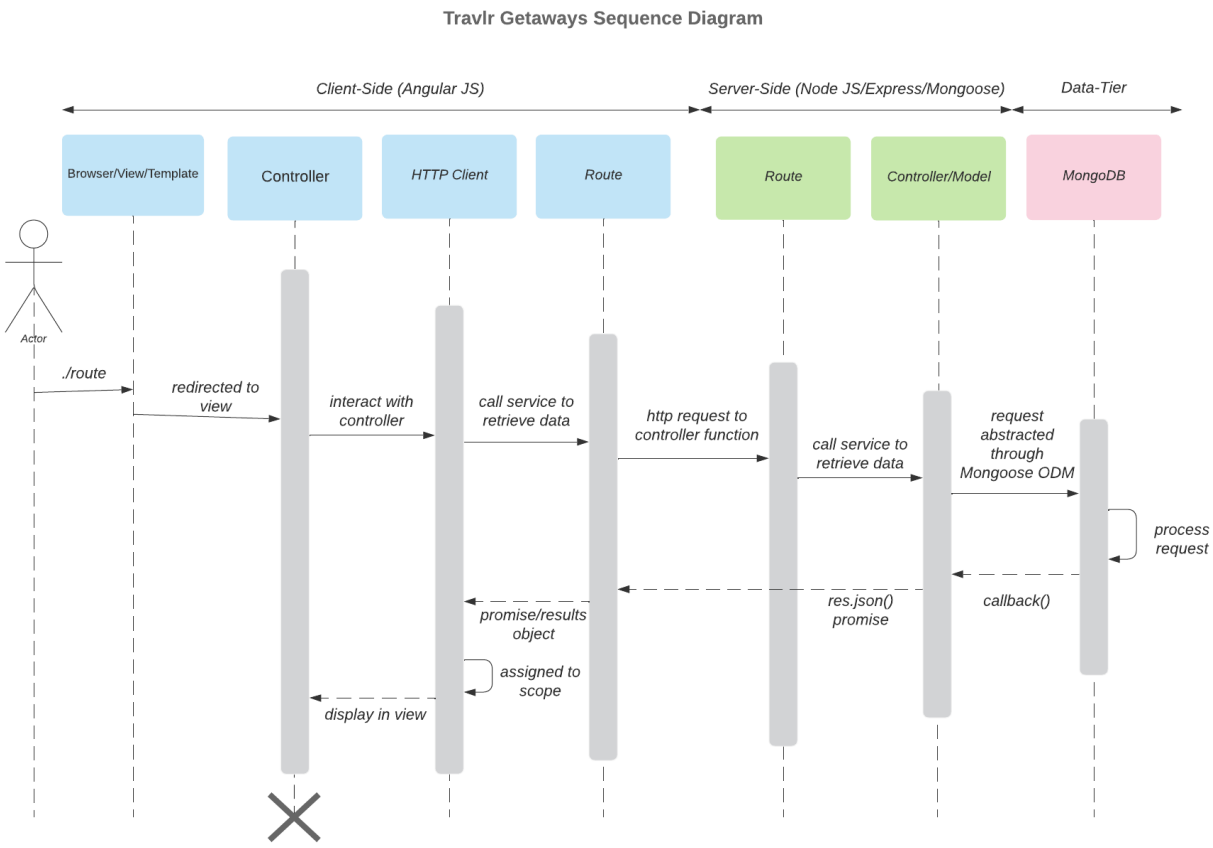
A text version of the component diagram is available: CS 465 Full Stack Component Diagram Text Version.

The component diagram for the Travlr Getaways system outlines the high-level structure of the application and shows how different modules interact to meet the business requirements. On the client side, the user interface is broken into reusable components including a traveler portfolio for browsing and booking trips and a client session manager to maintain state. These components are built to be dynamic and interactive, especially when extended into the Angular SPA for administrators.

On the server side, the architecture is anchored by the Express application, which functions as the routing hub and controller layer. It communicates with a session manager that handles authentication and protects sensitive routes. The server also connects to the Mongoose ODM (Object Document Mapper), which acts as the bridge between the API and the MongoDB database. Mongoose allows the application to define schemas and enforce structure on otherwise schema-less MongoDB documents. The database itself stores all critical records, including users, travel packages, and bookings, and supports efficient querying through indexes and document relationships.

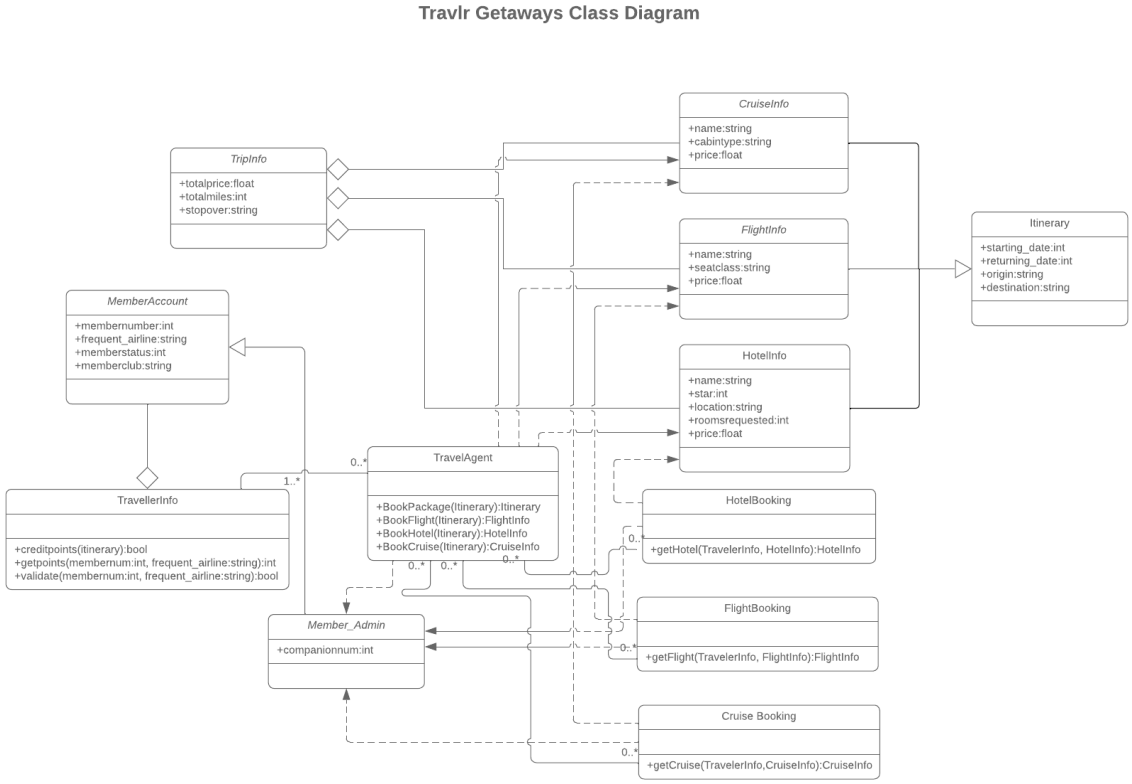
This system is designed to be modular and layered. The separation between the client, server, and database layers makes it easier to test, update, and expand the application as needed. The design supports current features like login, package browsing, and admin management, while leaving space for future features like reviews, ratings, and more complex filtering.

## Sequence Diagram




This sequence diagram shows how a user request flows through the Travlr Getaways web app. The user interacts with the browser, which triggers a route in Angular. The Angular component uses an HTTP client to request data from the server. The Express route handles the request and passes it to a controller, which queries MongoDB for trip data. The response is sent back through the same path and displayed in the browser.

## Class Diagram

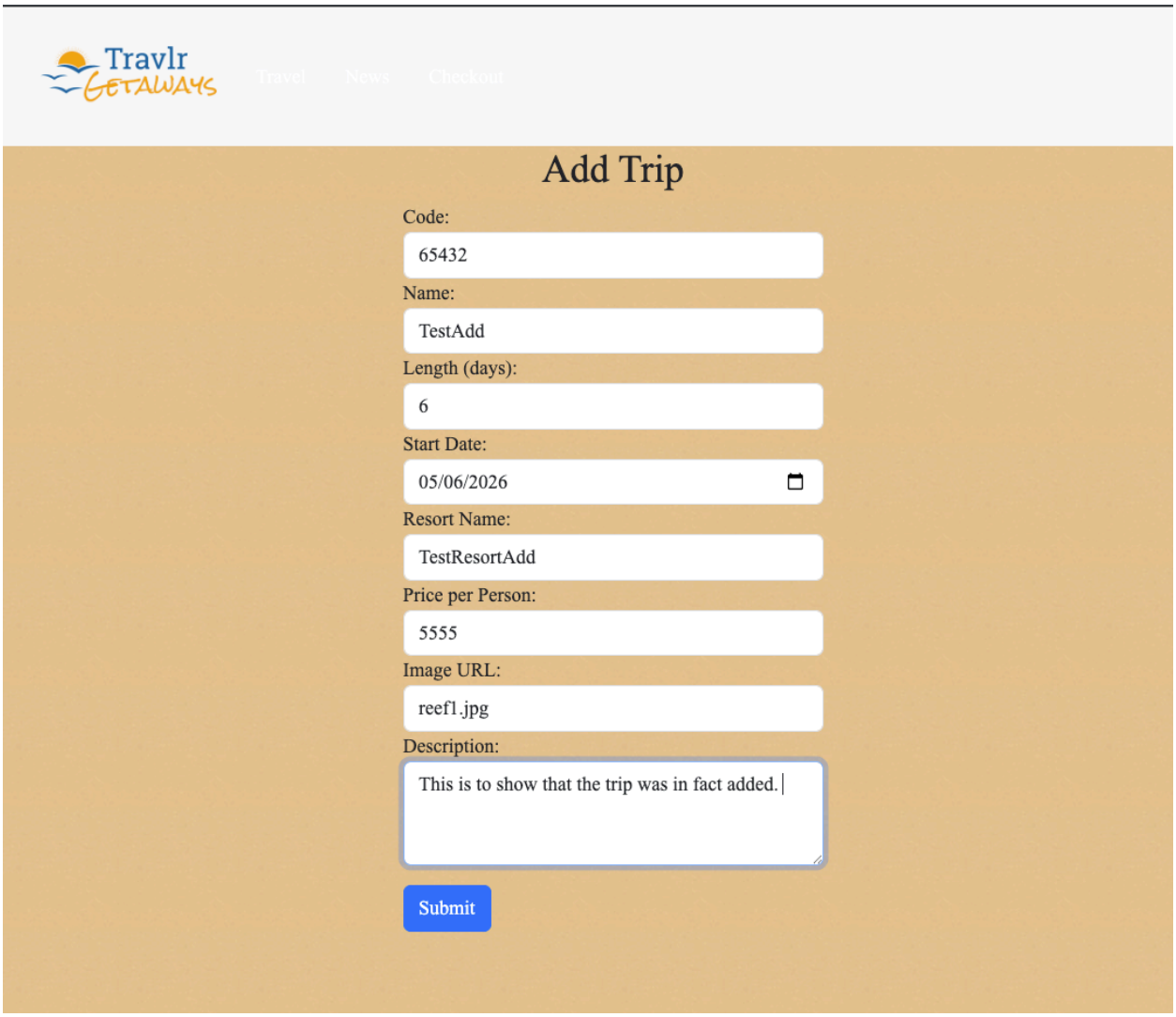



The class diagram illustrates the JavaScript classes used in the Travlr Getaways web application. It defines entities such as TravellerInfo, TripInfo, and TravelAgent, and connects them to booking components like HotelBooking, FlightBooking, and CruiseBooking. The diagram also outlines methods for booking travel services and managing member accounts. Relationships between classes reflect how user data, trip details, and booking functionality interact through a structured, object-oriented design.

## API Endpoints

| Method | Purpose | URL | Notes |
|---|---|---|---|
| **GET** | Retrieve all trips | /api/trips | Returns all trip entries stored in the MongoDB database in JSON format. |
| **GET** | Retrieve a single trip | /api/trips/:tripCode | Returns a single trip object identified by the tripcode parameter in the URL |

### The User Interface


Travlr
GETAWAYS
Travel
News
Checkout
Add Trip
Code:
65432
Name:
TestAdd
Length (days):
6
Start Date:
05/06/2026
Resort Name:
TestResortAdd
Price per Person:
5555
Image URL:
reef1.jpg
Description:
This is to show that the trip was in fact added.
Submit

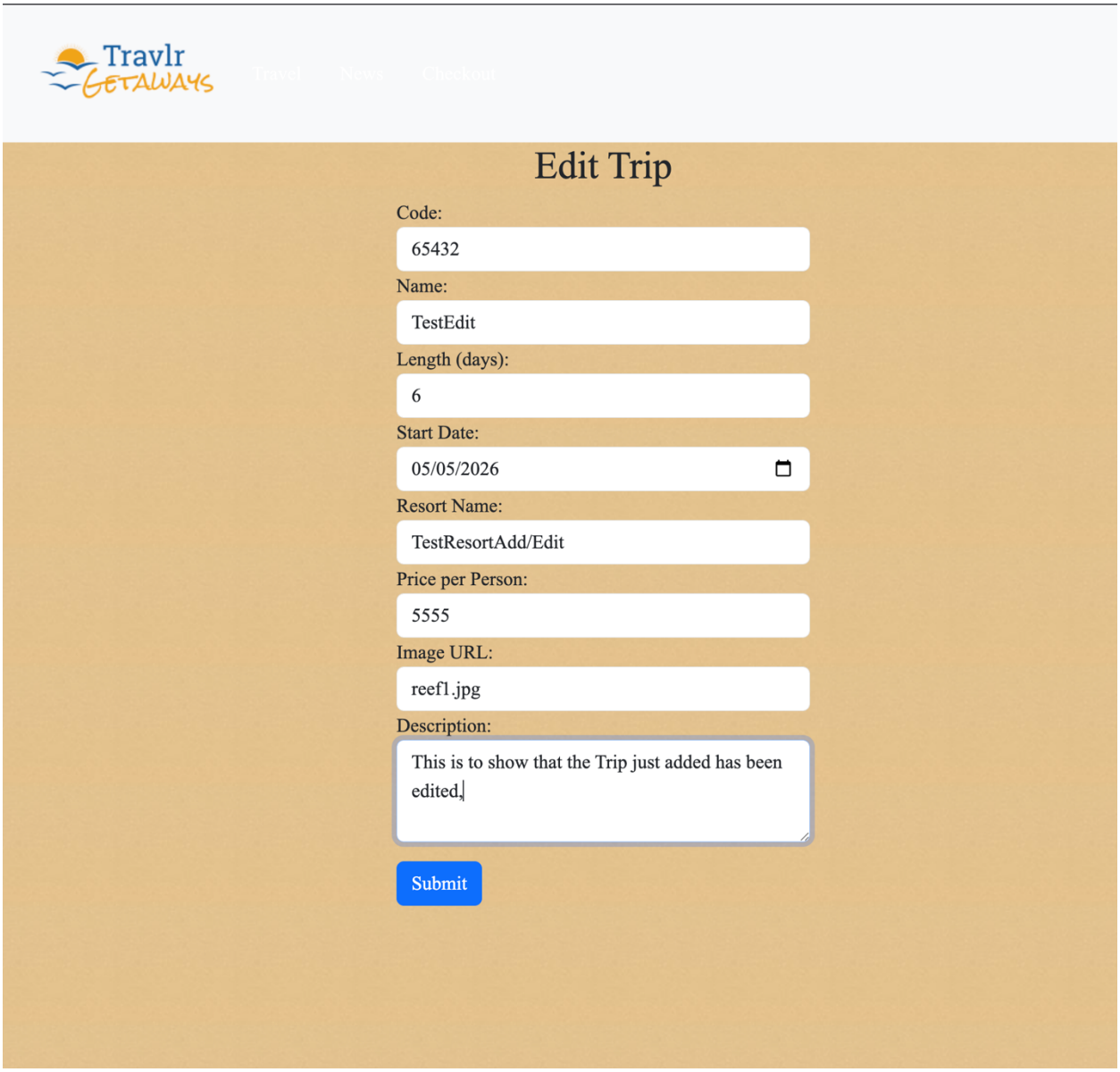
TestAdd

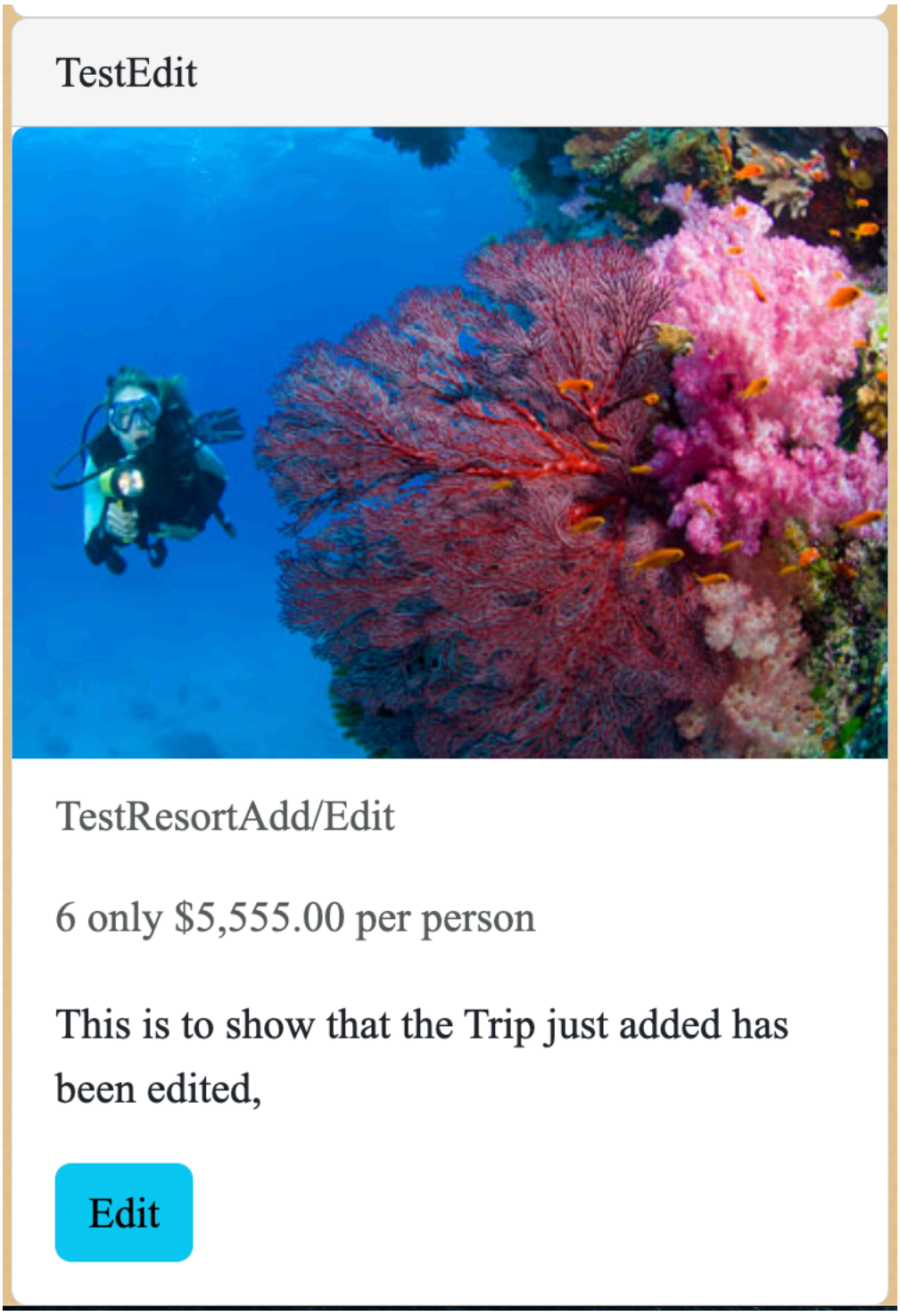
TestResortAdd

6 only $5,555.00 per person

This is to show that the trip was in fact added.

Edit


Travlr
GETAWAYS
Travel
News
Checkout
Edit Trip
Code:
65432
Name:
TestEdit
Length (days):
6
Start Date:
05/05/2026
Resort Name:
TestResortAdd/Edit
Price per Person:
5555
Image URL:
reef1.jpg
Description:
This is to show that the Trip just added has been edited.
Submit




The **Angular front-end** and the **Express back-end** follow different structural designs but are tightly integrated to deliver a modern, full-stack web application.

**Angular Project Structure (Client-Side SPA)**

The Angular project follows a **component-based architecture**, which breaks down the interface into reusable parts:

- src/app/components/ contains UI components such as trip cards or forms.
- src/app/services/ holds services that make HTTP requests to the API (e.g., TripDataService).
- src/app/models/ defines TypeScript interfaces for structured data (like Trip).
- Routing is handled client-side with the Angular Router (app-routing.module.ts), allowing seamless page transitions without reloads.

This structure enables the Angular app to function as a **Single Page Application (SPA)**. All rendering occurs client-side, and only data is exchanged with the back end. This offers a more dynamic and responsive user experience.

**Express Project Structure (Server-Side API)**

The Express app is organized in a **MVC-style pattern**:

- routes/ defines API endpoints (e.g., /api/trips).
- controllers/ contain the logic that handles the API requests (like retrieving or updating trips).
- models/ holds the Mongoose schemas that define MongoDB data structures.
- config/ stores configuration files like passport.js for authentication.

The Express server acts as a **RESTful API** provider. It does not render views for the SPA, but it handles all data-related operations securely.

**SPA vs. Traditional Web Application**

Compared to a traditional web app that reloads pages with every interaction, the Angular SPA:

- Loads once and dynamically updates the DOM based on user actions.
- Communicates with the API using **AJAX calls (via Angular's HttpClient)**, reducing server load and improving speed.
- Supports richer UI features such as form validation, animations, client-side routing, and real-time feedback.

This results in a smoother, app-like experience for users.

**Testing the SPA and API Integration**

To ensure the Angular SPA properly communicates with the Express API and database, the following steps are used:

1. **Start the servers**:
    - Launch the Express API (npm start) and ensure MongoDB is running.
    - Run the Angular app with ng serve.
2. **Test GET requests**:
    - Open the app in a browser and navigate to the trip listing or detail page.

    - Use Chrome DevTools (Network tab) to confirm a GET request is made to /api/trips.
    - Verify that the response data is correctly rendered in the view.
3. **Test PUT (or POST) requests**:
    - Use the app's form to edit or create a trip.
    - Submit the form and check the Network tab to confirm a PUT or POST request is sent.
    - Check the response and reload the data to verify the change is reflected in the UI and persisted in the database.
4. **Use Postman or Insomnia** to test the API endpoints separately for comparison and debugging.

This ensures the Angular front end and Express back end are successfully integrated and working together as intended.